

Terraform and CDK (CDKTF)

Deployed AWS Infrastructure with Multiple Programming Python + Terraform CDKTF

PROBLEM

I wanted to create an AWS S3 bucket using **Python + CDKTF** instead of writing Terraform HCL. I also needed versioning, encryption, public access block, and proper tags.

HOW I DEBUGGED IT

First, I checked my AWS access. I checked the Python and package versions first.

Instead of changing the system Python,

I created a **virtual environment (venv)** and installed the required packages there.

Normal/system Python → I tried it → it did NOT work

Python **venv** → I tried it → it WORKED. Then I tested the Python code and ran

cdktf synth

After that, I ran:

cdktf diff to make sure everything looked right before deployment.

THE SOLUTION

cdktf deploy

The S3 bucket was created successfully with versioning, encryption, public access blocked, and proper tags.

Has anyone else faced this problem?

Have you used Python + CDKTF for AWS infrastructure?

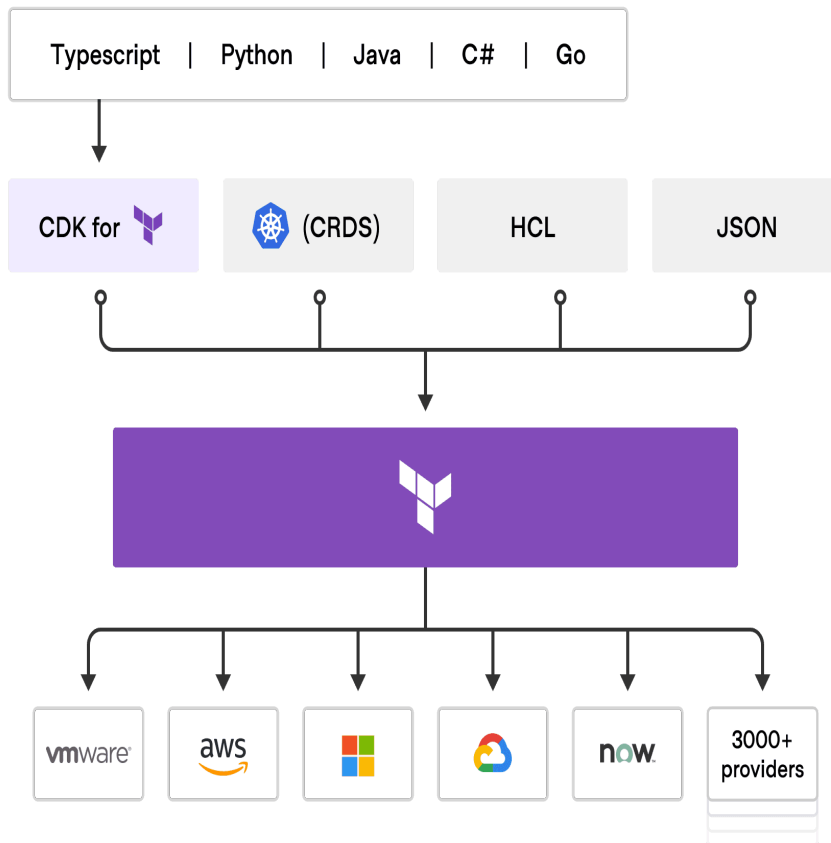
How did you fix it?

Note: All infrastructure images, official docs, and video links are included on the last page

Blog: https://vishnusai.hashnode.dev/terraform-and-cdk-cdktf?utm_source=hashnode&utm_medium=feed

Hashnode Profile: <https://hashnode.com/@vishnusaiK>

GitHub : <https://github.com/ksaivishnusaiKvs>



2. Prerequisites

Ubuntu EC2 instance, Python 3, pip, Node.js/npm, CDKTF CLI, Terraform CLI

AWS CLI

AWS credentials or an IAM role with permission to create the required S3 resources

Verify AWS Access

```
aws sts get-caller-identity
```

```
Apply Complete! Resources: 4 added, 0 changed, 0 destroyed.

No outputs found.
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$
aws sts get-caller-identity
{
  "UserId": "AIDAR3O2KSZSEGMV2DJNG",
  "Account": "127696279140",
  "Arn": "arn:aws:iam::127696279140:user/vishnusai"
}
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$
```

Step 1: Create infrastructure: Create the CDKTF Project

```
cd ~
```

```
rm -rf prod-s3
```

```
mkdir prod-s3
```

```
cd prod-s3
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
* Synthesizing  
ubuntu@ip-172-31-36-199:~/prod-s3$ cd ~  
rm -rf prod-s3  
mkdir prod-s3  
cd prod-s3  
ubuntu@ip-172-31-36-199:~/prod-s3$ python3 --version  
pip3 --version  
Python 3.12.3  
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
```

```
pip install --upgrade pip
```

```
pip install cdktf constructs cdktf-cdktf-provider-aws
```

```
source .venv/bin/activate  
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ pip install --upgrade pip  
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (24.0)  
Collecting pip  
  Using cached pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)  
  Using cached pip-26.2.1-py3-none-any.whl (1.8 MB)  
Installing collected packages: pip  
  Attempting uninstall: pip  
    Found existing installation: pip 24.0  
    Uninstalling pip-24.0:  
      Successfully uninstalled pip-24.0  
Successfully installed pip-26.2.1  
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ pip install cdktf constructs cdktf-cdktf-provider-aws  
Collecting cdktf  
  Downloading cdktf-0.21.0-py3-none-any.whl.metadata (1.3 kB)  
Collecting constructs  
  Downloading constructs-10.8.1-py3-none-any.whl.metadata (2.9 kB)  
Collecting cdktf-cdktf-provider-aws  
  Downloading cdktf_cdktf_provider_aws-21.22.1-py3-none-any.whl.metadata (4.1 kB)  
Collecting jsii<2.0.0,>=1.112.0 (from cdktf)  
  Downloading jsii-1.140.0-py3-none-any.whl.metadata (79 kB)  
Collecting publication>=0.0.3 (from cdktf)  
  Downloading publication-0.0.3-py2.py3-none-any.whl.metadata (8.7 kB)  
Collecting typeguard<4.3.0,>=2.13.3 (from cdktf)  
  Downloading typeguard-4.2.1-py3-none-any.whl.metadata (3.7 kB)  
Collecting attrs<27.0,>=21.2 (from jsii<2.0.0,>=1.112.0->cdktf)  
  Using cached attrs-26.1.0-py3-none-any.whl.metadata (8.8 kB)  
Collecting cattrs<27.0,>=1.8 (from jsii<2.0.0,>=1.112.0->cdktf)  
  Downloading cattrs-26.2.0-py3-none-any.whl.metadata (8.5 kB)  
Collecting typeguard<4.3.0,>=2.13.3 (from cdktf)  
  Downloading typeguard-2.13.3-py3-none-any.whl.metadata (3.6 kB)  
Collecting python-dateutil (from jsii<2.0.0,>=1.112.0->cdktf)  
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)  
Collecting typing_extensions<5.0,>=3.8 (from jsii<2.0.0,>=1.112.0->cdktf)  
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)  
Collecting six>=1.5 (from python-dateutil->jsii<2.0.0,>=1.112.0->cdktf)  
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)  
Downloading cdktf-0.21.0-py3-none-any.whl (2.4 MB)  
 2.4/2.4 MB 109.0 MB/s 0:00:00  
Downloading constructs-10.8.1-py3-none-any.whl (787 kB)  
 787.0/787.0 kB 46.4 MB/s 0:00:00  
Downloading jsii-1.140.0-py3-none-any.whl (503 kB)  
Downloading typeguard-2.13.3-py3-none-any.whl (17 kB)  
Using cached attrs-26.1.0-py3-none-any.whl (67 kB)  
Downloading cattrs-26.2.0-py3-none-any.whl (74 kB)  
Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)  
Downloading cdktf_cdktf_provider_aws-21.22.1-py3-none-any.whl (59.3 MB)  
 59.3/59.3 MB 172.0 MB/s 0:00:00  
Downloading publication-0.0.3-py2.py3-none-any.whl (7.7 kB)
```

STEP2: Create the CDKTF configuration

```
cat > cdktf.json <<'EOF'
{
  "language": "python",
  "app": "python3 main.py"
}
EOF
```

This tells CDKTF that the application entry point is main.py and that Python is the project language.

```
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages (from python-dateutil->jsii<2.0.0,>=1.112.0->cdktf) (1.17.0)
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cat > cdktf.json <<'EOF'
{
  "language": "python",
  "app": "python3 main.py"
}
EOF
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ nano main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 main.py
Traceback (most recent call last):
  File "/home/ubuntu/prod-s3/main.py", line 3, in <module>
    from cdktf_cdktf_provider_aws import AwsProvider
ImportError: cannot import name 'AwsProvider' from 'cdktf_cdktf_provider_aws' (/home/ubuntu/prod-s3/.venv/lib/python3.12/site-packages/cdktf_cdktf_provider_aws/__init__.py)
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cd ~/prod-s3
source .venv/bin/activate
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 -c "from cdktf_cdktf_provider_aws.provider import AwsProvider; print(AwsProvider)"
<class 'cdktf_cdktf_provider_aws.provider.AwsProvider'>
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 -c "from cdktf_cdktf_provider_aws.s3_bucket import S3Bucket; print(S3Bucket)"
<class 'cdktf_cdktf_provider_aws.s3_bucket.S3Bucket'>
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ nano main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ vi main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cdktf diff

: Synthesizing
```

```
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cat > cdktf.json <<'EOF'
{
  "language": "python",
  "app": "python3 main.py"
}
EOF
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ nano main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 main.py
Traceback (most recent call last):
  File "/home/ubuntu/prod-s3/main.py", line 3, in <module>
    from cdktf_cdktf_provider_aws import AwsProvider
ImportError: cannot import name 'AwsProvider' from 'cdktf_cdktf_provider_aws' (/home/ubuntu/prod-s3/.venv/lib/python3.12/site-packages/cdktf_cdktf_provider_aws/__init__.py)
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cd ~/prod-s3
source .venv/bin/activate
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 -c "from cdktf_cdktf_provider_aws.provider import AwsProvider; print(AwsProvider)"
<class 'cdktf_cdktf_provider_aws.provider.AwsProvider'>
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 -c "from cdktf_cdktf_provider_aws.s3_bucket import S3Bucket; print(S3Bucket)"
<class 'cdktf_cdktf_provider_aws.s3_bucket.S3Bucket'>
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ nano main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ vi main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cdktf diff
```

Step 3: Write code for infra Python Infrastructure Code. Create the infrastructure file

```
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cat main.py
from cdktf import App, TerraformStack
from constructs import Construct

from cdktf_cdktf_provider_aws.provider import AwsProvider
from cdktf_cdktf_provider_aws.s3_bucket import S3Bucket
from cdktf_cdktf_provider_aws.s3_bucket_versioning import S3BucketVersioningA
from cdktf_cdktf_provider_aws.s3_bucket_public_access_block import (
    S3BucketPublicAccessBlock
)
from cdktf_cdktf_provider_aws.s3_bucket_server_side_encryption_configuration import (
    S3BucketServerSideEncryptionConfigurationA
)

class ProdS3Stack(TerraformStack):

    def __init__(self, scope: Construct, stack_id: str):
        super().__init__(scope, stack_id)

        # AWS provider
        AwsProvider(
            self,
            "aws",
            region="ap-south-1"
        )

        # S3 bucket
        bucket = S3Bucket(
            self,
            "prod-bucket",
            bucket="kv-prod-s3-demo-20260909",
            tags={
                "Environment": "prod",
                "Application": "my-app",
                "ManagedBy": "cdktf"
            }
        )

        # Versioning
        S3BucketVersioningA(
            self,
            "versioning",
            bucket=bucket.id,
            versioning_configuration={
                "status": "Enabled"
            }
        )
```

```
        }
    }

    # Versioning
    S3BucketVersioningA(
        self,
        "versioning",
        bucket=bucket.id,
        versioning_configuration={
            "status": "Enabled"
        }
    )

    # Block public access
    S3BucketPublicAccessBlock(
        self,
        "public-access",
        bucket=bucket.id,
        block_public_acls=True,
        block_public_policy=True,
        ignore_public_acls=True,
        restrict_public_buckets=True
    )

    # Server-side encryption
    S3BucketServerSideEncryptionConfigurationA(
        self,
        "encryption",
        bucket=bucket.id,
        rule=[
            {
                "apply_server_side_encryption_by_default": [
                    {
                        "sse_algorithm": "AES256"
                    }
                ]
            }
        ]
    )

app = App()
ProdS3Stack(app, "prod-s3")
app.synth()
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ |
```

CODE :

```
from cdktf import App, TerraformStack
from constructs import Construct

from cdktf_cdktf_provider_aws.provider import AwsProvider
from cdktf_cdktf_provider_aws.s3_bucket import S3Bucket
from cdktf_cdktf_provider_aws.s3_bucket_versioning import S3BucketVersioningA
from cdktf_cdktf_provider_aws.s3_bucket_public_access_block import (
    S3BucketPublicAccessBlock
)
from cdktf_cdktf_provider_aws.s3_bucket_server_side_encryption_configuration import (
    S3BucketServerSideEncryptionConfigurationA
)

class ProdS3Stack(TerraformStack):

    def __init__(self, scope: Construct, stack_id: str):
        super().__init__(scope, stack_id)

        # AWS provider
        AwsProvider(
            self,
            "aws",
            region="ap-south-1"
        )

        # S3 bucket
        bucket = S3Bucket(
            self,
            "prod-bucket",
            bucket="kv-prod-s3-demo-20260909",
            tags={
                "Environment": "prod",
                "Application": "my-app",
                "ManagedBy": "cdktf"
            }
        )

        # Versioning
        S3BucketVersioningA(
            self,
            "versioning",
            bucket=bucket.id,
            versioning_configuration={
                "status": "Enabled"
            }
        )

        # Block public access
        S3BucketPublicAccessBlock(
            self,
            "public-access",
            bucket=bucket.id,
            block_public_acls=True,
            block_public_policy=True,
            ignore_public_acls=True,
            restrict_public_buckets=True
        )
```

```

    # Server-side encryption
    S3BucketServerSideEncryptionConfigurationA(
        self,
        "encryption",
        bucket=bucket.id,
        rule=[
            {
                "apply_server_side_encryption_by_default": [
                    {
                        "sse_algorithm": "AES256"
                    }
                ]
            }
        ]
    )

```

```
app = App()
```

```
ProdS3Stack(app, "prod-s3")
```

```
app.synth()
```

Step 4: What the Python Code Creates

AWS Provider: Uses ap-south-1 as the AWS region.

S3 Bucket: Creates the production bucket.

Tags: Adds Environment, Application, and ManagedBy tags for ownership and FinOps tracking.

Versioning: Keeps object versions so accidental changes/deletions can be recovered.

Public Access Block: Blocks public ACLs and public bucket policies.

Encryption: Enables S3 server-side encryption using AES256.

Step5: python3 main.py

Test the Python Code. If there is no Python traceback/error, the CDKTF application code is valid enough to synthesize.

```
⚡ Synthesizing
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cd ~/prod-s3
nano main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ vi main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ python3 main.py
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cdktf synth

Generated Terraform code for the stacks: prod-s3
```

Step6: cdktf synth

CDKTF converts the Python infrastructure definition into Terraform configuration under the generated CDKTF output.

```
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cdktf synth

Generated Terraform code for the stacks: prod-s3
```

Step7: cdktf diff

Review the resources before deployment. In the successful setup, the plan showed 4 resources to add and 0 to change or destroy.

```
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cdktf diff
prod-s3 Initializing the backend...
prod-s3 Successfully configured the backend "local"! Terraform will automatically
use this backend unless the backend configuration changes.
prod-s3 Initializing provider plugins...
- Finding hashicorp/aws versions matching "6.25.0"...
prod-s3 - Installing hashicorp/aws v6.25.0...
prod-s3 - Installed hashicorp/aws v6.25.0 (signed by HashiCorp)

Terraform has created a lock file ".terraform.lock.hcl" to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
prod-s3 Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
prod-s3 - Fetching hashicorp/aws 6.25.0 for linux_amd64...
prod-s3 - Retrieved hashicorp/aws 6.25.0 for linux_amd64 (signed by HashiCorp)
prod-s3 - Obtained hashicorp/aws checksums for linux_amd64; All checksums for this platform were already tracked in the lock file
prod-s3 Success! Terraform has validated the lock file and found no need for changes.
prod-s3 Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:
prod-s3 # aws_s3_bucket.prod-bucket (prod-bucket) will be created
+ resource "aws_s3_bucket" "prod-bucket" {
+   acceleration_status      = (known after apply)
+   acl                      = (known after apply)
+   arn                      = (known after apply)
+   bucket                  = "kv-prod-s3-demo-20260909"
+   bucket_domain_name      = (known after apply)
+   bucket_prefix           = (known after apply)
+   bucket_region           = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy           = false
+   hosted_zone_id          = (known after apply)
+   id                      = (known after apply)
+   object_lock_enabled      = (known after apply)
+   policy                  = (known after apply)
+   region                  = "ap-south-1"
+   request_payer            = (known after apply)
```



```

+ server_side_encryption_configuration (known after apply)
+ versioning (known after apply)
+ website (known after apply)
}

# aws_s3_bucket_public_access_block.public-access (public-access) will be created
+ resource "aws_s3_bucket_public_access_block" "public-access" {
+   block_public_acls      = true
+   block_public_policy    = true
+   bucket                 = (known after apply)
+   id                     = (known after apply)
+   ignore_public_acls     = true
+   region                 = "ap-south-1"
+   restrict_public_buckets = true
}

# aws_s3_bucket_server_side_encryption_configuration.encryption (encryption) will be created
+ resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {
+   bucket = (known after apply)
+   id     = (known after apply)
+   region = "ap-south-1"

+   rule {
+     blocked_encryption_types = []
+   }
}

# aws_s3_bucket_versioning.versioning (versioning) will be created
+ resource "aws_s3_bucket_versioning" "versioning" {
+   bucket = (known after apply)
+   id     = (known after apply)
+   region = "ap-south-1"

+   versioning_configuration {
+     mfa_delete = (known after apply)
+     status     = "Enabled"
+   }
}

Plan: 4 to add, 0 to change, 0 to destroy.

```

Saved the plan to: plan

To perform exactly these actions, run the following command to apply:

```
terraform apply "plan"
```

Step8: cdktf deploy

When CDKTF asks for approval, choose Approve/yes. Terraform then creates the AWS resources

```
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ cdktf deploy
prod-s3 Initializing the backend...
prod-s3 Initializing provider plugins...
prod-s3 - Reusing previous version of hashicorp/aws from the dependency lock file
prod-s3 - Using previously-installed hashicorp/aws v6.25.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
prod-s3 Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:
prod-s3 # aws_s3_bucket.prod-bucket (prod-bucket) will be created
+ resource "aws_s3_bucket" "prod-bucket" {
+   acceleration_status      = (known after apply)
+   acl                      = (known after apply)
+   arn                      = (known after apply)
+   bucket                   = "kv-prod-s3-demo-20260909"
+   bucket_domain_name       = (known after apply)
+   bucket_prefix            = (known after apply)
+   bucket_region            = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy            = false
+   hosted_zone_id           = (known after apply)
+   id                       = (known after apply)
+   object_lock_enabled       = (known after apply)
+   policy                   = (known after apply)
+   region                   = "ap-south-1"
+   request_payer             = (known after apply)
+   tags                     = {
+     "Application" = "my-app"
+     "Environment" = "prod"
+     "ManagedBy"  = "cdktf"
+   }
+   tags_all               = {
+     "Application" = "my-app"
+     "Environment" = "prod"
+     "ManagedBy"  = "cdktf"
+   }
}
```

Step 9: See the image below; infra was created with Terraform/CDKTF using Python, and the data was validated

```
+ resource "aws_s3_bucket_public_access_block" "public-access" {
  + block_public_acls      = true
  + block_public_policy    = true
  + bucket                = (known after apply)
  + id                    = (known after apply)
  + ignore_public_acls     = true
  + region                 = "ap-south-1"
  + restrict_public_buckets = true
}

# aws_s3_bucket_server_side_encryption_configuration.encryption (encryption) will be created
+ resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {
  + bucket = (known after apply)
  + id     = (known after apply)
  + region = "ap-south-1"

  + rule {
    + blocked_encryption_types = []
  }
}

# aws_s3_bucket_versioning.versioning (versioning) will be created
+ resource "aws_s3_bucket_versioning" "versioning" {
  + bucket = (known after apply)
  + id     = (known after apply)
  + region = "ap-south-1"

  + versioning_configuration {
    + mfa_delete = (known after apply)
    + status     = "Enabled"
  }
}
```

Plan: 4 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

prod-s3 Enter a value: yes

prod-s3 aws_s3_bucket.prod-bucket: Creating...

prod-s3 aws_s3_bucket.prod-bucket: Creation complete after 5s [id=kv-prod-s3-demo-20260909]

prod-s3 aws_s3_bucket_public_access_block.public-access: Creating...

aws_s3_bucket_versioning.versioning: Creating...

prod-s3 aws_s3_bucket_server_side_encryption_configuration.encryption: Creating...

prod-s3 aws_s3_bucket_public_access_block.public-access: Creation complete after 1s [id=kv-prod-s3-demo-20260909]

prod-s3 aws_s3_bucket_server_side_encryption_configuration.encryption: Creation complete after 1s [id=kv-prod-s3-demo-20260909]

prod-s3 aws_s3_bucket_versioning.versioning: Creation complete after 2s [id=kv-prod-s3-demo-20260909]

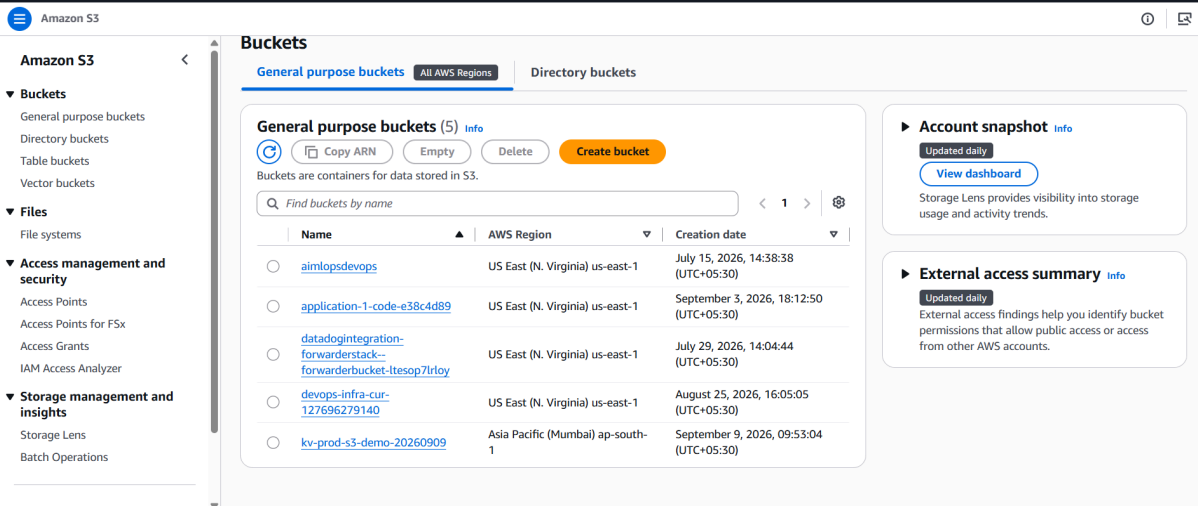
prod-s3

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Step 10: Verify whether the S3 Bucket `infra` is created or not in the cloud are not state file create not in local

```
app.synth()
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ ls -la
total 32
drwxrwxr-x 4 ubuntu ubuntu 4096 Sep  9 04:23 .
drwxr-x--- 54 ubuntu ubuntu 4096 Sep  9 04:41 ..
drwxrwxr-x 5 ubuntu ubuntu 4096 Sep  9 03:55 .venv
-rw-rw-r-- 1 ubuntu ubuntu 109 Sep  9 04:16 cdktf.json
drwxrwxr-x 3 ubuntu ubuntu 4096 Sep  9 04:18 cdktf.out
-rw-rw-r-- 1 ubuntu ubuntu 2078 Sep  9 04:18 main.py
-rw-rw-r-- 1 ubuntu ubuntu 6318 Sep  9 04:23 terraform.prod-s3.tfstate
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ aws s3 ls
2026-07-15 09:08:38 aimlopsdevops
2026-09-03 12:42:50 application-1-code-e38c4d89
2026-07-29 08:34:44 datadogintegration-forwarderstack--forwarderbucket-ltesop7lr1oy
2026-08-25 10:35:05 devops-infra-cur-127696279140
2026-09-09 04:23:04 kv-prod-s3-demo-20260909
(.venv) ubuntu@ip-172-31-36-199:~/prod-s3$ |
```

Step 11: Verify in the cloud ui the infrastructure resource is crete are not in cloud UI my s3 create throw CDKTF; name: s3 `kv-prod-s3-demo-20260909`



The screenshot shows the Amazon S3 console interface. On the left is a navigation menu with categories like Buckets, Files, Access management and security, and Storage management and insights. The main area is titled 'Buckets' and shows 'General purpose buckets (5)'. Below this is a table listing the buckets. The buckets listed are:

Name	AWS Region	Creation date
aimlopsdevops	US East (N. Virginia) us-east-1	July 15, 2026, 14:38:38 (UTC+05:30)
application-1-code-e38c4d89	US East (N. Virginia) us-east-1	September 3, 2026, 18:12:50 (UTC+05:30)
datadogintegration-forwarderstack--forwarderbucket-ltesop7lr1oy	US East (N. Virginia) us-east-1	July 29, 2026, 14:04:44 (UTC+05:30)
devops-infra-cur-127696279140	US East (N. Virginia) us-east-1	August 25, 2026, 16:05:05 (UTC+05:30)
kv-prod-s3-demo-20260909	Asia Pacific (Mumbai) ap-south-1	September 9, 2026, 09:53:04 (UTC+05:30)

On the right side of the console, there are two informational panels: 'Account snapshot' and 'External access summary', both with 'Updated daily' status and links to their respective dashboards.

TAGS show the Terraform file; compare the two images

Tags

You can use tags to track storage costs, organize general purpose buckets, and specify permissions for a general purpose bucket. AWS-generated tags are created by AWS and are read-only. [Learn more](#)

S3 Console now uses s3:ListTagsForResource, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets by default. To use these APIs for tagging, please provide permissions to s3:ListTagsForResource, s3:TagResource, and s3:UntagResource actions. [Learn more](#)

User-defined tags

AWS-generated tags

User-defined tags (3) [Info](#)

Delete

Add new tag

< 1 >

☐	Key	Value - optional ✎	☐
☐	Application	my-app	
☐	Environment	prod	
☐	ManagedBy	cdktf	

```
+ policy = (known after apply)
+ region = "ap-south-1"
+ request_payer = (known after apply)
+ tags = {
  + "Application" = "my-app"
  + "Environment" = "prod"
  + "ManagedBy" = "cdktf"
}
+ tags_all = {
  + "Application" = "my-app"
  + "Environment" = "prod"
  + "ManagedBy" = "cdktf"
}
+ website_domain = (known after apply)
+ website_endpoint = (known after apply)

+ cors_rule (known after apply)

+ grant (known after apply)
```

```
# aws_s3_bucket_public_access_block.public-access (public-access) will be created
+ resource "aws_s3_bucket_public_access_block" "public-access" {
+   block_public_acls       = true
+   block_public_policy     = true
+   bucket                  = (known after apply)
+   id                      = (known after apply)
+   ignore_public_acls     = true
+   region                  = "ap-south-1"
+   restrict_public_buckets = true
+ }

# aws_s3_bucket_server_side_encryption_configuration.encryption (encryption) will be created
+ resource "aws_s3_bucket_server_side_encryption_configuration" "encryption" {
+   bucket = (known after apply)
+   id     = (known after apply)
+   region = "ap-south-1"

+   rule {
+     blocked_encryption_types = []
+   }
+ }

# aws_s3_bucket_versioning.versioning (versioning) will be created
+ resource "aws_s3_bucket_versioning" "versioning" {
+   bucket = (known after apply)
+   id     = (known after apply)
+   region = "ap-south-1"

+   versioning_configuration {
+     mfa_delete = (known after apply)
+     status     = "Enabled"
+   }
+ }

Plan: 4 to add, 0 to change, 0 to destroy.
```

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Access Points

Bucket overview

AWS Region

Asia Pacific (Mumbai) ap-south-1

Amazon Resource Name (ARN)

 arn:aws:s3:::kv-prod-s3-demo-20260909

Creation date

September 9, 2026, 09:53:04 (UTC+05:30)

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)

Disabled

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Access Points

Permissions overview

Access finding

Access findings are provided by IAM external access analyzers. Learn more about [How IAM analyzer findings work](#)

[View analyzer for ap-south-1](#)

Block public access (bucket settings)

Edit

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

 On

► Individual Block Public Access settings for this bucket

Default encryption

[Edit](#)

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [Info](#)

Server-side encryption with Amazon S3 managed keys (SSE-S3)

Bucket Key

When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering calls to AWS KMS. [Learn more](#)

Disabled

Blocked encryption types [Info](#)

Server-Side Encryption with Customer-Provided Keys (SSE-C)

REFDOC:

Hashicorp Official : <https://developer.hashicorp.com/terraform/cdktf>

AWS: <https://aws.amazon.com/blogs/opensource/announcing-cdk-for-terraform-on-aws/>

installsteps: 1: <https://spacelift.io/blog/terraform-cdk>

2: <https://dev.to/aws-builders/cdk-for-terraform-cdktf-on-aws-how-to-configure-an-s3-remote-backend-and-deploy-a-lambda-function-url-using-python-3okk>

CDKTF official HashiCorp channel: <https://www.youtube.com/watch?v=Y9suaDVT-SY&t=27s>

<https://developer.hashicorp.com/terraform/tutorials/cdktf/cdktf-install>

DOCsteps: <https://spacelift.io/blog/terraform-cdk>

